Chapter 2
Introduction to C Programming

C How to Program, 9/e, GE

Objectives

In this chapter, you'll:

- Write simple C programs.
- Use simple input and output statements.
- Use the fundamental data types.
- Learn computer memory concepts.
- Use arithmetic operators.
- Learn the precedence of arithmetic operators.
- Write simple decision-making statements.

2.1 Introduction

- The C language facilitates a structured and disciplined approach to computer program design.
- Introduce C programming and present several examples that illustrate many important features of C.
- In Chapters 3 and 4 we present an introduction to structured programming in C.

2.2 A Simple C Program: Printing a Line of Text

- begin with `//`, indicating that these two lines are comments.
- Comments document programs and improve program readability.
- Comments do not cause the computer to perform any action when the program is run.

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution.
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

```
Welcome to C!
```

Fig. 2.1 | A first program in C.

You can also use `/*...*/` multi-line comments

The `void` in parentheses here means that `main` does not receive any information.

The backslash (\) is called an escape character.

The escape sequence `\n` means newline.

Every statement must end with a semicolon (also known as the statement terminator)

`printf` causes the computer to perform an action.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

- Comments are ignored by the C compiler and do not cause any machine-language object code to be generated.
- Comments also help other people read and understand your program.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

#include Preprocessor Directive

`#include <stdio.h>`

- is a directive to the C preprocessor.

- Lines beginning with # are processed by the preprocessor **before** compilation.
- Line 3 tells the preprocessor to include the contents of the standard input/output header (<stdio.h>) in the program.
- This header contains information used by the compiler when compiling calls to standard input/output library functions such as `printf`.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

Blank Lines and White Space

- You use blank lines, space characters and tab characters (i.e., "tabs") to make programs easier to read.
- Together, these characters are known as white space. White-space characters are normally ignored by the compiler.

The `main` Function

- **`int main( void )`**
  - is a part of every C program.
  - The parentheses after `main` indicate that `main` is a program building block called a function.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

- C programs contain one or more functions, one of which *must* be `main`.
- Every program in C begins executing at the function `main`.
- The keyword **`int`** to the left of `main` indicates that `main` "returns" an integer (whole number) value.
- We'll explain what it means for a function to "return a value" when we demonstrate how to create your own functions in Chapter 5.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

- For now, simply include the keyword **`int`** to the left of `main` in each of your programs.
- Functions also can receive information when they're called upon to execute.
- The `void` in parentheses here means that `main` does not receive any information.

Good Programming Practice 2.1

Every function should be preceded by a comment describing the function's purpose.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

- A left brace, {, begins the body of every function
- A corresponding right brace ends each function
- This pair of braces and the portion of the program between the braces is called a block.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

An Output Statement

```
printf( "Welcome to C!\n" );
```

- instructs the computer to perform an action, namely to print on the screen the string of characters marked by the quotation marks.
- A string is sometimes called a character string, a message or a literal.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

- The entire line, including the `printf` function (the "f" stands for "formatted"), its argument within the parentheses and the semicolon (`;`), is called a statement.
- Every statement must end with a semicolon (also known as the statement terminator).

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

```
Welcome to C!
```

- When the preceding `printf` statement is executed, it prints the message `Welcome to C!` on the screen.
- The characters normally print exactly as they appear between the double quotes in the `printf` statement.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

Escape Sequences

- Notice that the characters `\n` were not printed on the screen.
- The backslash (\) is called an escape character.
- It indicates that `printf` is supposed to do something out of the ordinary.
- When encountering a backslash in a string, the compiler looks ahead at the next character and combines it with the backslash to form an escape sequence.

- The escape sequence \n means newline.
- When a newline appears in the string output by a `printf`, the newline causes the cursor to position to the beginning of the next line on the screen.
- Some common escape sequences are listed in Fig. 2.2.

Escape sequence	Description
\n	Newline. Position the cursor at the beginning of the next line.
\t	Horizontal tab. Move the cursor to the next tab stop.
\a	Alert. Produces a sound or visible alert without changing the current cursor position.
\\	Backslash. Insert a backslash character in a string.
\"	Double quote. Insert a double-quote character in a string.

Fig. 2.2 | Some common escape sequences .

2.2 A Simple C Program: Printing a Line of Text (Cont.)

```
1  // Fig. 2.1: fig02_01.c
2  // A first program in C.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     printf( "Welcome to C!\n" );
9  } // end function main
```

- Because the backslash has special meaning in a string, i.e., the compiler recognizes it as an escape character, we use a **double** backslash (****) to place a single backslash in a string.
- Printing a double quote also presents a problem because double quotes mark the boundaries of a string—such **quotes** are **not** printed.
- By using the escape sequence **\"** in a string to be output by `printf`, we indicate that `printf` should display a double quote.
- The right brace, `}`, indicates that the end of `main` has been reached.

Good Programming Practice 2.2

Add a comment to the line containing the right brace, `}`, that closes every function, including `main`.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

The Linker and Executables

- Standard library functions like `printf` and `scanf` are **not** part of the C programming language.
  - For example, the compiler cannot find a spelling error in `printf` or `scanf`.
    - When the compiler compiles a `printf` statement, it merely provides space in the object program for a "call" to the library function.
  - But the **compiler** does not know where the library functions are—the **linker** does.
- When the **linker** runs, it locates the library functions and inserts the proper calls to these library functions in the object program.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

- Now the object program is complete and ready to be executed.
- For this reason, the **linked** program is called an executable.
- If the function name is **misspelled**, it's the linker that will spot the **error**, because it will not be able to match the name in the C program with the name of any known function in the libraries.

2.2 A Simple C Program: Printing a Line of Text (Cont.)

Using Multiple `printf`s

```c
// Fig. 2.3: fig02_03.c
// Printing on one line with two printf statements.
#include <stdio.h>

// function main begins program execution
int main( void )
{
   printf( "Welcome " );
   printf( "to C!\n" );
} // end function main
```

```
Welcome to C!
```

ig. 2.3 | Printing on one line with two printf statements.

```c
// Fig. 2.4: fig02_04.c
// Printing multiple lines with a single printf.
#include <stdio.h>

// function main begins program execution
int main( void )
{
   printf( "Welcome\nto\nC!\n" );
} // end function main
```

```
Welcome
to
C!
```

Fig. 2.4 | Printing multiple lines with a single printf.

Program Demo 2.2

2.3 Another Simple C Program: Adding Two Integers

- Uses the Standard Library function `scanf` to obtain two integers typed by a user at the keyboard, computes the sum of these values and prints the result using `printf`.

```
1  // Fig. 2.5: fig02_05.c
2  // Addition program.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     int integer1; // first number to be entered by user
9     int integer2; // second number to be entered by user
10    int sum; // variable in which sum will be stored
11
12    printf( "Enter first integer\n" ); // prompt
13    scanf( "%d", &integer1 ); // read an integer
14
15    printf( "Enter second integer\n" ); // prompt
16    scanf( "%d", &integer2 ); // read an integer
17
18    sum = integer1 + integer2; // assign total to sum
19
20    printf( "Sum is %d\n", sum ); // print sum
21 } // end function main
```

This message is called a prompt because it tells the user to take a specific action.

Uses `scanf` to obtain a value from the user.

```
Enter first integer
45
Enter second integer
72
Sum is 117
```

Fig. 2.5 | Addition program.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

Variables and Variable Definitions

```
int integer1; // first number to be entered by user
int integer2; // second number to be entered by user
int sum; // variable in which sum will be stored
```

are definitions.

- The names `integer1`, `integer2` and `sum` are the names of variables—locations in memory where values can be stored for use by a program.
- These definitions specify that the variables `integer1`, `integer2` and `sum` are of type `int`, which means that they'll hold integer values, i.e., whole numbers such as 7, –11, 0, 31914 and the like.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

- All variables must be defined with a name and a data type before they can be used in a program.
- The preceding definitions could have been combined into a single definition statement as follows:

```
int integer1, integer2, sum;
```

but that would have made it difficult to describe the variables with corresponding comments

2.3 Another Simple C Program: Adding Two Integers (Cont.)

Identifiers and Case Sensitivity

- A variable name in C is any valid identifier.
- An identifier is **a series of characters** consisting of letters, digits and underscores (_) that does ***not*** begin with a digit.
- C is case sensitive—uppercase and lowercase letters are different in C, so `a1` and `A1` are different identifiers.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

Prompting Messages

```
printf( "Enter first integer\n" ); // prompt
```

- displays the literal "`Enter first integer`" and positions the cursor to the beginning of the next line.
- This message is called a prompt because it **tells** the user to take a specific action.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

The `scanf` Function and Formatted Inputs

- The next statement

```
scanf( "%d", &integer1 ); // read an integer
```

  uses scanf to obtain a value from the user.
- The `scanf` function reads from the standard input, which is usually the keyboard.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

- This `scanf` has two arguments, `"%d"` and `&integer1`.
- The first, the format control string, indicates the **type** of data that should be input by the user.
  - The %d conversion specifier indicates that the data should be an integer (the letter **d** stands for "decimal integer").
  - The % in this context is treated by `scanf` (and `printf` as we'll see) as a special character that begins a conversion specifier.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

- The second argument of `scanf` begins with an ampersand (&)—called the address operator in C—followed by the variable name.
  - The &, when combined with the variable name, tells `scanf` the location (or address) in memory at which the variable `integer1` is stored.
  - The computer then stores the value that the user enters for `integer1` at that location.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

- When the computer executes the preceding `scanf`, it waits for the user to enter a value for variable `integer1`.
- The user responds by typing an integer, then pressing the *Enter* key to send the number to the computer.
- The computer then assigns this number, or value, to the variable `integer1`.
- Any subsequent references to `integer1` in this program will use this same value.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

```
printf( "Enter second integer\n" ); // prompt
```

- displays the message `Enter second integer` on the screen, then positions the cursor to the beginning of the next line.
- This `printf` also prompts the user to take action.

```
scanf( "%d", &integer2 ); // read an integer
```

- obtains a value for variable `integer2` from the user.

Functions `printf` and `scanf` facilitate **interaction** between the user and the computer.

Because this interaction resembles a dialogue, it's often called interactive computing.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

Assignment Statement

- The assignment statement

```
sum = integer1 + integer2; // assign total to sum
```

calculates the total of variables `integer1` and `integer2` and assigns the result to variable `sum` using the assignment operator =.

  - The statement is read as, "`sum` *gets* the value of `integer1 + integer2`." Most calculations are performed in **assignments**.
  - The = operator and the + operator are called binary operators because each has two operands.
  - The + operator's two operands are `integer1` and `integer2`.
  - The = operator's two operands are `sum` and the value of the expression `integer1 + integer2`.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

Printing with a Format Control String

```
printf( "Sum is %d\n", sum ); // print sum
```

- calls function `printf` to print the literal `Sum is` followed by the numerical value of variable `sum` on the screen.
- This `printf` has two arguments, `"Sum is %d\n"` and `sum`.
  - The first argument is the format control string.
    - It contains some literal characters to be displayed, and it contains the conversion specifier `%d` indicating that an integer will be printed.
  - The second argument specifies the value to be printed.

2.3 Another Simple C Program: Adding Two Integers (Cont.)

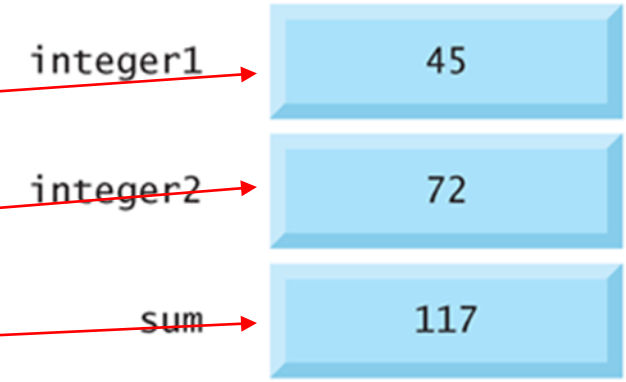
Calculations in `printf` Statements

- We could have **combined** the previous two statements into the statement
  - `printf( "Sum is %d\n", integer1 + integer2 );`
- The right brace, }, at line 21 indicates that the end of function `main` has been reached.

Program Demo 2.3

2.4 Memory Concepts

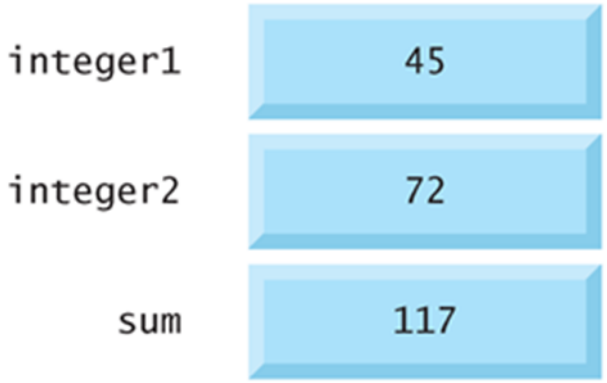
```
1  // Fig. 2.5: fig02_05.c
2  // Addition program.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     int integer1; // first number to be entered by user
9     int integer2; // second number to be entered by user
10
11    printf( "Enter first integer\n" ); // prompt
12    scanf( "%d", &integer1 ); // read an integer
13
14    printf( "Enter second integer\n" ); // prompt
15    scanf( "%d", &integer2 ); // read an integer
16
17    int sum; // variable in which sum will be stored
18    sum = integer1 + integer2; // assign total to sum
19
20    printf( "Sum is %d\n", sum ); // print sum
21 } // end function main
```



- Variable names such as `integer1`, `integer2` and `sum` actually correspond to locations in the computer's memory.
- Every variable has a name, a type and a value.

2.4 Memory Concepts (Cont.)

```
1  // Fig. 2.5: fig02_05.c
2  // Addition program.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main( void )
7  {
8     int integer1; // first number to be entered by user
9     int integer2; // second number to be entered by user
10
11    printf( "Enter first integer\n" ); // prompt
12    scanf( "%d", &integer1 ); // read an integer
13
14    printf( "Enter second integer\n" ); // prompt
15    scanf( "%d", &integer2 ); // read an integer
16
17    int sum; // variable in which sum will be stored
18    sum = integer1 + integer2; // assign total to sum
19
20    printf( "Sum is %d\n", sum ); // print sum
21 } // end function main
```



- This occurs when the calculated total of `integer1` and `integer2` is placed into location `sum` (destroying the value already in `sum`).
- Thus, when a value is read from a memory location, the process is said to be nondestructive.

2.5 Arithmetic in C

C operation	Arithmetic operator	Algebraic expression	C expression
Addition	+	$f + 7$	`f + 7`
Subtraction	–	$p - c$	`p - c`
Multiplication	*	bm	`b * m`
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	`x / y`
Remainder	%	$r \bmod s$	`r % s`

Fig. 2.9 | Arithmetic operators.

Integer Division and the Remainder Operator

- Integer division yields an integer result
  - For example, the expression `7 / 4` evaluates to `1` and the expression `17 / 5` evaluates to `3`
- C provides the remainder operator, `%`, which yields the remainder after integer division
  - Can be used only with integer operands
- The expression `x % y` yields the remainder after `x` is divided by `y`.
  - Thus, `7 % 4` yields `3` and `17 % 5` yields `2`

2.5 Arithmetic in C (Cont.)

Parentheses for Grouping Subexpressions

- **Parentheses** are used in C expressions in the same manner as in algebraic expressions.
- For example, to multiply `a` times the quantity `b + c` we write `a * ( b + c )`.

2.5 Arithmetic in C (Cont.)

Rules of Operator Precedence

- C applies the operators in arithmetic expressions in a precise sequence determined by the following rules of operator precedence, which are generally the same as those in algebra:

Operator(s)	Operation(s)	Order of evaluation (precedence)
( )	Parentheses	Evaluated first. If the parentheses are nested, the expression in the *innermost* pair is evaluated first. If there are several pairs of parentheses "on the same level" (i.e., not nested), they're evaluated left to right.
*	Multiplication	Evaluated second. If there are several, they're evaluated left to right.
/	Division	
%	Remainder	
+	Addition	Evaluated third. If there are several, they're evaluated left to right.
-	Subtraction	
=	Assignment	Evaluated last.

Fig. 2.10 | Precedence of arithmetic operators.

2.5 Arithmetic in C (Cont.)

- Figure 2.11 illustrates the order in which the operators are applied.

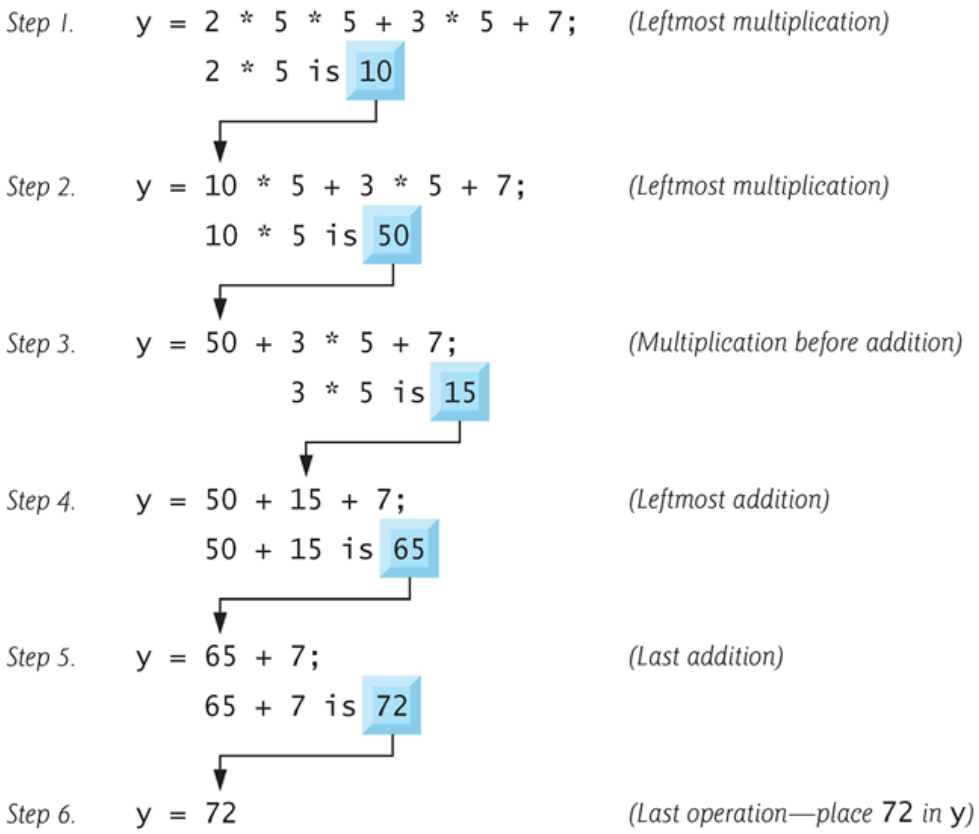


Fig. 2.11 | Order in which a second-degree polynomial is evaluated.

2.6 Decision Making: Equality and Relational Operators

- We might make a decisions in a program.
  - To determine
  - whether a person's grade on an exam is greater than or equal to 60
  - whether the program should print the message "Congratulations! You passed."

  - If the condition is true (i.e., the condition is met) the statement in the body of the `if` statement is executed.
  - If the condition is false (i.e., the condition isn't met) the body statement is not executed.

Algebraic equality or relational operator	C equality or relational operator	Example of C condition	Meaning of C condition
Relational operators			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y
Equality operators			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y

Fig. 2.12 | Equality and relational operators.

- The **relational** operators all have the same level of precedence and they associate left to right.
- The **equality** operators have a lower level of precedence than the relational operators and they also associate left to right.
- In C, a condition may actually be *any expression that generates a **zero** (false) or nonzero (**true**) value*.

2.6 Decision Making: Equality and Relational Operators

- Uses six `if` statements to compare two numbers entered by the user.

```
1  // Fig. 2.13: fig02_13.c
2  // Using if statements, relational
3  // operators, and equality operators.
4  #include <stdio.h>
5
6  // function main begins program execution
7  int main( void )
8  {
9     printf( "Enter two integers, and I will tell you\n" );
10    printf( "the relationships they satisfy: " );
11
12    int num1; // first number to be read from user
13    int num2; // second number to be read from user
14
15    scanf( "%d %d", &num1, &num2 ); // read two integers
16
17    if ( num1 == num2 ) {
18       printf( "%d is equal to %d\n", num1, num2 );
19    } // end if
20
21    if ( num1 != num2 ) {
22       printf( "%d is not equal to %d\n", num1, num2 );
23    } // end if
24
25    if ( num1 < num2 ) {
26       printf( "%d is less than %d\n", num1, num2 );
27    } // end if
28
29    if ( num1 > num2 ) {
30       printf( "%d is greater than %d\n", num1, num2 );
31    } // end if
32
33    if ( num1 <= num2 ) {
34       printf( "%d is less than or equal to %d\n", num1, num2 );
35    } // end if
36
37    if ( num1 >= num2 ) {
38       printf( "%d is greater than or equal to %d\n", num1, num2 );
39    } // end if
40 } // end function main
```

```
Enter two integers, and I will tell you
the relationships they satisfy: 3 7
3 is not equal to 7
3 is less than 7
3 is less than or equal to 7
```

```c
// Fig. 2.13: fig02_13.c
// Using if statements, relational
// operators, and equality operators.
#include <stdio.h>

// function main begins program execution
int main( void )
{
   printf( "Enter two integers, and I will tell you\n" );
   printf( "the relationships they satisfy: " );

   int num1; // first number to be read from user
   int num2; // second number to be read from user

   scanf( "%d %d", &num1, &num2 ); // read two integers

   if ( num1 == num2 ) {
      printf( "%d is equal to %d\n", num1, num2 );
   } // end if

   if ( num1 != num2 ) {
      printf( "%d is not equal to %d\n", num1, num2 );
   } // end if

   if ( num1 < num2 ) {
      printf( "%d is less than %d\n", num1, num2 );
   } // end if

   if ( num1 > num2 ) {
      printf( "%d is greater than %d\n", num1, num2 );
   } // end if

   if ( num1 <= num2 ) {
      printf( "%d is less than or equal to %d\n", num1, num2 );
   } // end if

   if ( num1 >= num2 ) {
      printf( "%d is greater than or equal to %d\n", num1, num2 );
   } // end if
} // end function main
```

```
Enter two integers, and I will tell you
the relationships they satisfy: 22 12
22 is not equal to 12
22 is greater than 12
22 is greater than or equal to 12
```

```
Enter two integers, and I will tell you
the relationships they satisfy: 7 7
7 is equal to 7
7 is less than or equal to 7
7 is greater than or equal to 7
```

Operators	Associativity
()	left to right
* / %	left to right
+ -	left to right
< <= > >=	left to right
== !=	left to right
=	right to left

Fig. 2.14 | Precedence and associativity of the operators discussed so far.

Good Programming Practice 2.12

Refer to the operator precedence chart when writing expressions containing many operators. Confirm that the operators in the expression are applied in the proper order. If you're uncertain about the order of evaluation in a complex expression, use parentheses to group expressions or break the statement into several simpler statements. Be sure to observe that some of C's operators such as the assignment operator (=) associate from right to left *rather than from left to right.*

2.6 Decision Making: Equality and Relational Operators

- Some of the words we've used in the C programs in this chapter—in particular `int` and `if`—are keywords or reserved words of the language.
- These words have special meaning to the C compiler, so you must be careful not to use these as identifiers such as variable names.

Keywords

`auto`	`do`	`goto`	`signed`	`unsigned`
`break`	`double`	`if`	`sizeof`	`void`
`case`	`else`	`int`	`static`	`volatile`
`char`	`enum`	`long`	`struct`	`while`
`const`	`extern`	`register`	`switch`	
`continue`	`float`	`return`	`typedef`	
`default`	`for`	`short`	`union`	

Keywords added in C99 standard

`_Bool` `_Complex` `_Imaginary` `inline` `restrict`

Keywords added in C11 standard

`_Alignas` `_Alignof` `_Atomic` `_Generic` `_Noreturn` `_Static_assert` `_Thread_local`

Fig. 2.15 | C's keywords.

Program Demo 2.6

2.7 Secure C Programming

- CERT C Secure Coding Standard
  - Guidelines that help you avoid programming practices that open systems to attacks
- Avoid Single-Argument `printfs`
  - If you need to display a string that terminates with a newline, use the `puts` function, which displays its string argument followed by a newline character
  - For example

```
printf( "Welcome to C!\n" );
```

should be written as:

```
puts( "Welcome to C!" );
```

  - We did not include `\n` in the preceding string because puts adds it automatically.

2.7 Secure C Programming (cont.)

- If you need to display a string without a terminating newline character, use `printf` with two arguments.
  - For example

```
printf( "Welcome " );
```

should be written as:

```
printf( "%s", "Welcome " );
```

  - These changes are responsible coding practices that eliminate certain security C vulnerabilities as we get deeper into C

Program Exercise

2.32 ***(Body Mass Index Calculator)*** We introduced the body mass index (BMI) calculator in Exercise 1.12. The formulas for calculating BMI are

$$BMI = \frac{weightInPounds \times 703}{heightInInches \times heightInInches}$$

or

$$BMI = \frac{weightInKilograms}{heightInMeters \times heightInMeters}$$

Create a BMI calculator application that reads the user's weight in pounds and height in inches (or, if you prefer, the user's weight in kilograms and height in meters), then calculates and displays the user's body mass index. Also, the application should display the following information from the Department of Health and Human Services/National Institutes of Health so the user can evaluate his/her BMI:

```
BMI VALUES
Underweight: less than 18.5
Normal:      between 18.5 and 24.9
Overweight:  between 25 and 29.9
Obese:       30 or greater
```

```
Please enter your height (in inches): 72
Please enter your weight (in pounds): 200
Your BMI is 27

BMI VALUES
Underweight:    less than 18.5
Normal:         between 18.5 and 24.9
Overweight:     between 25 and 29.9
Obese:          30 or greater
```

[*Note:* In this chapter, you learned to use the `int` type to represent whole numbers. The BMI calculations when done with `int` values will both produce whole-number results. In Chapter 4 you'll learn to use the `double` type to represent numbers with decimal points. When the BMI calculations are performed with `double`s, they'll both produce numbers with decimal points—these are called "floating-point" numbers.]